

CEG 4424/6424 Security Attacks and Defenses

Homework #4

Student Name: Bhuwan Pandey
Wright UID: U01155118
Undergraduate/Graduate: Graduate

Question 1 (10 points) Modern operating systems have incorporated a bit “non_executable” to label whether the content in a memory page (e.g., a memory block) can be executed. If stack frames are organized into these memory pages and labeled as “non_executable”.

(1) (5 points) Will this stop the shellcode **injected to the stack** (e.g., the buffer allocated in the stack) from being executed? Please justify.

Answer: Yes, the non-executable bit prevents execution of code stored in memory regions marked as non-executable, such as the stack. Therefore, traditional buffer overflow attacks that inject shellcode into the stack and attempt to execute it will fail because the CPU will block execution from that region. Meaning, it does not prevent the overflow itself, only the execution of injected code.

(2) (5 Points, Question for Graduate Students) Will this eradicate the possibility that an attacker executes unwanted code in the target system?

Answer: No, it does not eliminate all possibilities. Attackers can still use techniques such as return-to-libc attacks, return-oriented programming (ROP). These methods reuse existing executable code in memory instead of injecting new code, bypassing the “non-executable bit” protection.

Question 2 (10 points) We use a 32-bit operating system. Its int types, memory addresses, and registers, such as ESP, EBP and EIP, are represented by 32 bits. We compile a program that calls this function in this operating system.

```
int foo(int x) {  
  
    int y, z;  
    y = x+3;  
    z = x * 2;  
    return z;  
  
}
```

Please plot the stack frame of foo function before it returns.

Answer: Before the foo function returns, the stack frame plot looks something like this:

Return Address

Old EBP

x (argument)

$y = x + 3$

$z = x * 2$

Question 3 (20 points) We use a 32-bit operating system. Its int types, memory addresses, and registers, such as ESP, EBP and EIP, are represented by 32 bits. We compile a program that calls this function in this operating system.

```
void foo(int **input, int par){
    int tag = par;
    int *arr[10];
    printf("%x", tag);
    if(tag > 1){
        while(*input != 0x0000)
        {
            *arr = *input;
            arr++;
            input++;
        }
    }
    printf("%x", tag);
}
```

(1) (10 points) Please plot the stack frame of foo function before it returns.

Answer: The stack frame of foo function before it returns looks like:

Return Address

Old EBP

input (pointer)

par

tag

arr[10] (array)

(2) (10 Points, Question for Graduate Students) Suppose you design a buffer overflow attack that successfully launches a shell by exploiting the foo function, after this successful BOF attack, will the two "printf" commands yield the same output? Please justify.

Answer: No, not necessarily, because in a buffer overflow, stack memory (including tag) may be overwritten, and also control flow is altered (like jumping to shellcode). So first printf may print correct value whereas second may print modified/corrupted value, or may not execute at all.

Question 4 (10 points) Alice can read the file A, can read and write the file B, and cannot access the file C. Bob can read and write the file A, can execute the file B, and can read the file C.

1. Write a set of access control lists for this situation.

Answer: For file A,

Alice: Read
Bob: Read, Write

For file B:

Alice: Read, Write
Bob: Execute

For file C:

Alice: No Access
Bob: Read

2. Write a set of capability lists for this situation.

Answer: For Alice:

A: Read
B: Read, Write

And for Bob:

A: Read, Write
B: Execute
C: Read

Question 5 (20 points)

What is ransomware? Please give the basic introduction of three types of ransomware after reading the paper (The answers to this question should be at least half a page. You should summarize the content in your own words.)

Answer: Ransomware is a type of malicious software that restricts access to a user's system or data, typically by encrypting files, and demands payment (usually in crypto currency) to restore access. It has become a major cyber security threat due to its financial impact and widespread use.

One common type is crypto ransomware, such as CryptoLocker or WannaCry, which encrypts user files using strong cryptographic algorithms like RSA or AES. Once encryption is complete, victims must pay a ransom to obtain the decryption key. This type is highly effective because without the key, recovery is nearly impossible.

Another type is locker ransomware, which does not encrypt files but instead locks users out of their system entirely, often by displaying a full-screen message. While less destructive than crypto ransomware, it still disrupts access and demands payment.

A more advanced type is boot level ransomware, such as Petya, which infects the Master Boot Record (MBR) and prevents the operating system from loading. This makes the system unusable and increases the pressure on victims to pay quickly.

The analysis of WannaCry shows that ransomware often uses system cryptographic libraries, spreads via network vulnerabilities, and communicates with command-and-control servers through anonymized networks like Tor. Preventive measures include keeping systems updated, using antivirus software, and avoiding suspicious downloads. Overall, ransomware continues to evolve, making proactive security practices essential.